HTTPie CLI — Developer Guide

HTTPie (`http` / `https`) is a command-line HTTP client for working with APIs. You type requests in a short, readable syntax, and responses come back formatted and colorized. This guide is for developers who are new to the `httpie/cli` repository and need to use it, run it from source, and find their way around the code.

- **Version in this checkout:** 3.2.4 (`httpie/__init__.py`, released 2024-11-01)
- **Language:** Python 3 (`python_requires >= 3.7`; CI tests 3.7–3.12 on Linux, macOS and Windows)
- **License:** BSD
- **Built on:** `requests` (HTTP), `Pygments` (syntax colors), `rich` (help, progress bars), `multidict`, `requests-toolbelt` (multipart), `defusedxml`, `charset_normalizer`

The full user manual is `docs/README.md` (published at https://httpie.io/docs/cli). This guide summarizes it and adds the developer view.

---

1. Quick start

Run from source

```
git clone https://github.com/httpie/cli.git && cd cli
make all                    # creates ./venv, installs deps + editable httpie, runs tests
source venv/bin/activate    # prompt shows (httpie)
http --version              # -> 3.2.4
```

`make install` only sets things up and does not run tests. You can also skip activating the venv and call `./venv/bin/http` directly.

Windows (no `make`):

```
python -m venv --prompt httpie venv
venv\Scripts\activate
python -m pip install --upgrade -e .[dev]
```

First requests

```
http pie.dev/get                          # GET (default when there is no body)
http pie.dev/post name=Ada lang=py        # POST JSON {"name":"Ada","lang":"py"}
http PUT pie.dev/put X-API-Token:123 n:=5 # explicit method, header, raw-JSON number
http :3000/health                         # localhost shorthand -> http://localhost:3000/health
https example.org                         # `https` defaults the scheme to https://
```

---

2. Installed commands

`setup.cfg` defines three console scripts:

Command	Entry point	Purpose
`http`	`httpie.__main__:main`	Main HTTP client. Default scheme `http://`.
`https`	`httpie.__main__:main`	Same program. Default scheme `https://` (chosen by program name).
`httpie`	`httpie.manager.__main__:main`	Manager: plugins, sessions upgrade, update checks.

`python -m httpie` works the same as `http`.

---

3. Using the CLI

3.1 Synopsis

```
http [flags] [METHOD] URL [REQUEST_ITEM ...]
```

- **METHOD** is optional. HTTPie uses `GET` when there is no request body and `POST` when there is data (from data items or stdin).
- **URL**: the scheme defaults to `http://` (or `--default-scheme`). Shorthands: `:3000/x` → `http://localhost:3000/x`, `:/x` → `http://localhost/x`.
- **REQUEST_ITEM**: `key<separator>value` tokens that build the request (see below).

3.2 Request items (separators)

Syntax	Meaning	Example
`Header:Value`	HTTP header	`Authorization:'Bearer x'`
`Header;`	Header with an empty value	`Accept;`
`Header:`	Remove a default header	`User-Agent:`
`Header:@file`	Header value read from a file	`Cookie:@cookie.txt`
`name==value`	URL query parameter	`q==httpie page==2`
`name==@file`	Query parameter read from a file	`q==@query.txt`
`field=value`	String data field (JSON or form)	`name=Ada`
`field:=json`	Raw JSON value (number, bool, list, object, null)	`age:=36 tags:='["a","b"]'`
`field=@file`	String field read from a file	`bio=@bio.txt`
`field:=@file.json`	Raw JSON read from a file	`cfg:=@config.json`
`field@path`	File upload (implies `--form` multipart)	`cv@~/cv.pdf`
`field@path;type=x`	File upload with an explicit MIME type	`img@a.png;type=image/png`

Escape a separator with a backslash, e.g. `'foo\==bar'`. Use `--` to stop flag parsing.

Nested JSON. Data fields accept path syntax:

```
http --offline pie.dev/post \
  user[name]=Ada user[langs][]=py user[langs][]=c \
  user[age]:=36 items[0][id]:=1
# -> {"user":{"name":"Ada","langs":["py","c"],"age":36},"items":[{"id":1}]}
```

3.3 Body serialization

Flag	Effect
(default) / `-j`, `--json`	Data items are sent as a JSON object. `Content-Type: application/json`.
`-f`, `--form`	`application/x-www-form-urlencoded`, or multipart if any `field@file` items are present.
`--multipart`	Always multipart, even without files. `--boundary=X` sets the boundary.
`--raw 'text'`	Sends the given string as the body unchanged.
stdin	`echo '{"a":1}' | http POST url` — redirected input becomes the body.
`@file` (as an item on its own)	`http POST url @body.xml` sends the file contents as the body.
`-x`, `--compress`	Deflate-compresses the body (`-xx` compresses even if it doesn't help).
`--chunked`	Uses chunked transfer encoding.

3.4 Output control

By default HTTPie prints the response headers and body (`hb`). When stdout is redirected, it prints only the body (`b`). With `--offline` it prints the request (`HB`).

Flag	Prints
`-p`, `--print=WHAT`	Any combination of `H` request headers, `B` request body, `h` response headers, `b` response body, `m` metadata (elapsed time)
`-h`, `--headers`	Response headers only (`-p h`)
`-b`, `--body`	Response body only (`-p b`)
`-m`, `--meta`	Response metadata only (`-p m`)

Flag	Prints
`-v`, `--verbose`	Request and response (`HBhb`); `-vv` also adds `m`
`--all`	Also prints intermediary requests/responses (redirects)
`-P`, `--history-print=WHAT`	Like `--print`, applied to intermediary messages
`-q`, `--quiet`	No stdout/stderr except errors and warnings; `-qq` hides warnings too
`-S`, `--stream`	Streams the body line by line (like `tail -f`)
`-o`, `--output FILE`	Writes output to a file instead of stdout

Formatting:

Flag	Effect
`--pretty=all|colors|format|none`	Formatting/coloring. The default is `all` on a TTY and `none` when redirected.
`-s`, `--style=NAME`	Color theme (default `auto`; e.g. `monokai`, `fruity`, `solarized`)
`--sorted` / `--unsorted`	Turns sorting of headers and JSON keys on or off
`--format-options=...`	For example `json.indent:2,json.sort_keys:false,headers.sort:false,xml.indent:4`
`--response-charset=ENC`	Overrides the response encoding for display
`--response-mime=TYPE`	Overrides the MIME type used for coloring and formatting

3.5 Downloads

```
http -d pie.dev/image/png              # saves to a file named from Content-Disposition/URL
http -d -o out.png pie.dev/image/png   # explicit filename
http -dco big.iso https://example.org/big.iso   # resume (-c needs -o)
```

`--download` implies `--follow`. Shows a progress bar on stderr.

3.6 Authentication

```
http -a user:pass pie.dev/basic-auth/user/pass          # basic (default -A basic)
http -a user pie.dev/basic-auth/user/pass               # password prompt
http -A digest -a user:pass pie.dev/digest-auth/auth/user/pass
http -A bearer -a TOKEN pie.dev/bearer
http --ignore-netrc example.org                         # ignore ~/.netrc credentials
```

The built-in auth types are `basic`, `digest` and `bearer`. Plugins can add more (`httpie cli plugins install httpie-<name>`).

3.7 Sessions

Sessions keep headers, cookies and auth between requests.

```
http --session=dev -a user:pass pie.dev/get     # creates/updates named session "dev" (per host)
http --session=dev pie.dev/headers              # reuses auth + cookies
http --session=./s.json pie.dev/get             # anonymous session at an explicit path
http --session-read-only=dev pie.dev/get        # reads it but doesn't save changes
```

Named sessions are stored in `<config-dir>/sessions/<host_port>/<name>.json`. After an upgrade that changes the session layout, run `httpie cli sessions upgrade[-all]`.

3.8 Network and TLS options

Flag	Purpose
`-F`, `--follow`	Follows 30x redirects
`--max-redirects=N`	Redirect limit (default 30)
`--timeout=SECS`	Connection/read timeout
`--proxy=http:http://host:port`	Per-protocol proxy (`socks5://` is supported). The `HTTP(S)_PROXY` env vars also work.
`--verify=no|/path/ca.pem`	Turns off TLS verification or sets a CA bundle

Flag	Purpose
`--cert`, `--cert-key`, `--cert-key-pass`	Client certificate
`--ssl=tls1.2` / `--ciphers=...`	Protocol version / OpenSSL cipher list
`--path-as-is`	Doesn't collapse `/../` in the URL
`--max-headers=N`	Limits the number of response headers
`--offline`	Builds and prints the request without sending it
`--check-status`	Exits non-zero on 3xx/4xx/5xx (see exit codes)
`-I`, `--ignore-stdin`	Doesn't read stdin (**use this in scripts and cron jobs**)
`--default-scheme=https`	Scheme used when the URL has none
`--debug` / `--traceback`	Diagnostic info / full tracebacks
`--help` / `--manual` / `--version`	Help text, full manual (man page or pager), version

Any default flag can be turned off with `--no-<flag>` (e.g. `--no-style`, `--no-session`).

3.9 Exit codes

From `httpie/status.py`:

Code	Meaning
0	Success
1	Generic error (bad args, connection error, etc.)
2	Request timed out
3	HTTP 3xx (only with `--check-status` and without `--follow`)
4	HTTP 4xx (with `--check-status` or `--download`)
5	HTTP 5xx (with `--check-status` or `--download`)
6	Too many redirects
7	Plugin error
130	Interrupted (Ctrl-C)

Example for scripts:

```
if http --check-status --ignore-stdin --timeout=2.5 HEAD pie.dev/get &>/dev/null; then
  echo OK
else
  echo "failed with $?"
fi
```

3.10 Configuration

The config directory is chosen in this order (`httpie/config.py`):

1. `$HTTPIE_CONFIG_DIR`, if set
2. Windows: `%APPDATA%\httpie`
3. Legacy `~/.httpie`, if it exists
4. `$XDG_CONFIG_HOME/httpie`, which defaults to `~/.config/httpie`

`config.json` supports:

```
{
  "default_options": ["--style=fruity", "--follow"],
  "plugins_dir": "/custom/path/to/plugins"
}
```

`default_options` are added before every invocation's arguments.

3.11 The `httpie` manager command

```
httpie cli check-updates
httpie cli export-args                # dumps the CLI option spec as JSON
```

```
httpie cli sessions upgrade HOST NAME [--bind-cookies]
httpie cli sessions upgrade-all [--bind-cookies]
httpie cli plugins install httpie-foo  # also: list, upgrade, uninstall
                                       # (alias: `httpie plugins ...`)
```

---

4. Architecture

4.1 Request lifecycle

```
http …  ──►  httpie/__main__.py:main()
               └─► core.main() ─► core.raw_main()
                     • decode args, run daemon tasks (update checks)
                     • load plugins from plugins_dir
                     • prepend config default_options
                     • cli/definition.py `parser` ─► HTTPieArgumentParser.parse_args()
                         (cli/argparser.py: method/URL inference, request items
                          via cli/requestitems.py, nested JSON via cli/nested_json/)
                     • error handling → ExitStatus
               └─► core.program(args, env)
                     • client.collect_messages()  – generator
                         – sessions.get_httpie_session() (if --session)
                         – make_request_kwargs / make_send_kwargs
                         – build_requests_session() (adapters.py, ssl_.py, transport plugins)
                         – yields PreparedRequest, then Response(s); follows redirects
                         – saves session afterwards
                     • for each message: output/writer.write_message()
                         – output/streams.py picks a stream (raw/encoded/pretty/buffered)
                         – output/processing.py runs formatters + converters
                     • downloads.Downloader for --download
                     • status.http_status_to_exit_status() for --check-status
```

`Environment` (`httpie/context.py`) wraps stdin/stdout/stderr, whether they are TTYs, the config, and colors. Tests replace it with `MockEnvironment`.

4.2 Source map

Path	What lives there
`httpie/__main__.py`	Console entry point; maps Ctrl-C to exit code 130
`httpie/core.py`	`main`, `raw_main` (arg parsing + error handling), `program` (request/output loop)
`httpie/cli/definition.py`	**All CLI flags** declared via `ParserSpec` (also used for `--help`, man pages, `export-args`)
`httpie/cli/options.py`	`ParserSpec` / `Argument` abstraction over argparse
`httpie/cli/argparser.py`	`HTTPieArgumentParser`: post-processing, method guessing, stdin, auth
`httpie/cli/constants.py`	Separators, `--print` letters, pretty map, default format options
`httpie/cli/requestitems.py`	Turns `KeyValueArgs` into headers/data/params/files
`httpie/cli/nested_json/`	Tokenizer/parser/interpreter for `a[b][]=c` syntax
`httpie/client.py`	Builds and sends requests (`collect_messages`)
`httpie/sessions.py`	Persistent sessions (headers, cookies, auth)
`httpie/config.py`	Config dir resolution, `config.json`
`httpie/context.py`	`Environment`, `LogLevel`
`httpie/models.py`	Request/response wrappers, `OutputOptions`
`httpie/output/`	Writer, streams, processing, `formatters/` (colors, headers, json, xml), `lexers/`, `ui/` (rich help, palette, man pages, progress)
`httpie/downloads.py`	`--download` / `--continue`
`httpie/uploads.py`	Streaming, chunked and compressed request bodies
`httpie/ssl_.py`, `adapters.py`	TLS adapter (versions, ciphers, client certs)
`httpie/plugins/`	Plugin base classes, built-ins (basic/digest/bearer), plugin manager, registry
`httpie/manager/`	The `httpie` command (`cli.py` defines subcommands; `tasks/` implements them)
`httpie/internal/`	Background update checks (daemon), build channel
`httpie/legacy/`	Migrations for older session formats

Path	What lives there
`httpie/status.py`	`ExitStatus` enum
`tests/`	pytest suite (`test_*.py`), `utils/` helpers, `fixtures/`
`docs/README.md`	The full user manual (published to httpie.io)
`docs/installation/`, `docs/packaging/`	Install-method generator and packaging recipes (brew, snap, choco, deb/rpm…)
`extras/`	Shell completions (bash, fish), man pages, profiling/benchmarks, scripts

4.3 Plugins

Plugins are Python packages registered through entry points (`httpie/plugins/manager.py`):

Entry-point group	Base class (`httpie/plugins/base.py`)	Hook
`httpie.plugins.auth.v1`	`AuthPlugin`	`get_auth(username, password)` → `requests.auth.AuthBase`
`httpie.plugins.transport.v1`	`TransportPlugin` (`prefix`)	`get_adapter()` → `requests` adapter
`httpie.plugins.converter.v1`	`ConverterPlugin`	`supports(mime)`, `convert(body)`
`httpie.plugins.formatter.v1`	`FormatterPlugin`	`format_headers`, `format_body`, `format_metadata`

Minimal auth plugin:

```
from httpie.plugins import AuthPlugin

class MyAuthPlugin(AuthPlugin):
    name = 'My auth'
    auth_type = 'my-auth'          # used as: http -A my-auth -a creds URL
    def get_auth(self, username=None, password=None):
        return MyRequestsAuth(username, password)

# setup.cfg of the plugin package
[options.entry_points]
httpie.plugins.auth.v1 =
    my_auth = my_pkg:MyAuthPlugin
```

---

5. Development workflow

5.1 Make targets

Target	Does
`make` / `make list-tasks`	Lists tasks
`make all`	Uninstalls httpie, then runs install and test
`make install`	Creates `./venv` and installs `.[dev]`, `.[test]` and an editable httpie
`make test`	`python -m pytest`
`make test-cover`	Tests with coverage (`--cov=httpie --cov=tests`)
`make codestyle`	flake8 on `httpie/ tests/ extras/profiling/ docs/packaging/brew/ *.py`
`make test-dist`	Builds and installs the sdist and wheel, then runs `http --version`
`make test-all`	clean + install + test + test-dist + codestyle
`make doc-check`	markdownlint (`mdl`) on the docs
`make build`	Builds the sdist and wheel into `dist/` (build channel `pip`)
`make clean`	Removes the venv, build artifacts and caches

5.2 Tests

- Runner: **pytest** (config in `setup.cfg [tool:pytest]` and `pytest.ini`). Test paths are `httpie` and `tests`. Doctests are on (`--doctest-modules`) and `xfail_strict = True`.

- Most tests use the local `httpbin` fixture from `pytest-httpbin`, so they need no network. A few use `remote_httpbin` and are skipped when it isn't reachable.
- Main helper: `tests/utils/__init__.py::http(*args, ...)`. It runs the CLI in-process with a `MockEnvironment` and returns a `StrCLIResponse`/`BytesCLIResponse` with `.json`, `.stderr` and `.exit_status`.

```
from .utils import http, HTTP_OK

def test_get(httpbin):
    r = http('GET', httpbin + '/get', 'q==1')
    assert HTTP_OK in r
    assert r.json['args'] == {'q': '1'}

python -m pytest tests/test_uploads.py
python -m pytest tests/test_uploads.py::TestMultipartFormDataFileUpload::test_upload_ok
python -m pytest -k nested_json
```

5.3 Benchmarks

```
python extras/profiling/run.py         # compares your branch against master (`--fresh` for a clean master)
```

5.4 Contribution rules (from CONTRIBUTING.md)

- Check existing issues first. If your change alters behavior or the interface, discuss it before you start.
- **Code changes must include tests.** Changes to behavior must include docs (`docs/README.md`).
- Follow PEP 8. `make codestyle` must pass (E501 and W503 are ignored).
- Add an entry to `CHANGELOG.md`.
- In bug reports, include the output of `http --debug <args>`.
- CI (GitHub Actions, `.github/workflows/`) runs tests, code style, coverage, docs checks and packaging tests.

5.5 Adding a new CLI flag (checklist)

1. Declare it in `httpie/cli/definition.py` with `short_help` and `help`.
2. If it needs validation or derived values, post-process it in `httpie/cli/argparser.py`.
3. Use it in `client.py` (request), `output/` (display) or `core.py` (flow).
4. Add tests in `tests/` and document it in `docs/README.md`. Man pages and `export-args` are generated from the spec.
5. Add a `CHANGELOG.md` entry.

---

6. Troubleshooting

Symptom	Fix
`http` hangs in a script or cron job	It is waiting for stdin. Add `--ignore-stdin` (`-I`).
Sent a POST when you wanted a GET	Data items or piped stdin make the method POST. Pass `GET` explicitly.
Numbers/booleans sent as strings	Use `:=` instead of `=` (`n:=5`, `ok:=true`).
Colors missing when piping	Expected: `--pretty` defaults to `none` when stdout isn't a TTY. Use `--pretty=all`.
SSL errors against a dev server	`--verify=no` or `--verify=/path/ca.pem`
Script can't tell 404 from 200	Add `--check-status` and check the exit code.
Old session file warnings	`httpie cli sessions upgrade-all`
Need details for a bug report	`http --debug …` (versions, environment, plugins, request kwargs)
`which http` points to the system install	Activate `venv` or call `./venv/bin/http`.

---

7. Further reading

- `docs/README.md` — complete user manual
- `CONTRIBUTING.md` — contributor workflow
- `CHANGELOG.md` — release history
- `SECURITY.md` — how to report vulnerabilities
- https://httpie.io/docs/cli · https://pie.dev (a test API used throughout the docs)